Brief Announcement: Sinkless Orientation Made Trivial

Alexandre Nolin[*†‡]
Télécom SudParis
Palaiseau, France
alexandre.nolin@telecom-sudparis.eu

Abstract

We give a simple deterministic algorithm for Sinkless Orientation in Congest achieving optimal complexity $\Theta(\log_\delta n)$ on poly(n)-node graphs of minimum degree $\delta \geq 3$. Combined with the shattering argument by Ghaffari and Su [8, SODA 2017] for this problem, our result also implies an optimal randomized algorithm running in $\Theta(\log_\delta \log n)$ Congest rounds. Other known reductions imply corollaries for more balanced orientations, such as ones where nodes have at least $\lfloor \deg/3 \rfloor$ incoming and outgoing edges. Our algorithm is arguably much simpler than the previous algorithm achieving optimal complexity in Local.

CCS Concepts

• **Theory of computation → Distributed algorithms**.

Keywords

CONGEST model, sinkless orientation

ACM Reference Format:
Alexandre Nolin. 2026. Brief Announcement: Sinkless Orientation Made Trivial . In *ACM Symposium on Principles of Distributed Computing (PODC '26), July 6–10, 2026, Egham, United Kingdom.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3796701.3815953

1 Introduction

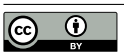
In the Local model of distributed computing [11], a network is modeled by a graph $G = (V, E)$, whose number of nodes we denote by $n = |V|$. Each node $v \in V$ is a computing entity, tasked with outputting its own small part of the solution to a problem Π, and is given a unique $O(\log n)$-bit identifier ID_v. To compute said solution, nodes can communicate over the edges of the graph. A round of communication in Local allows each node to send an arbitrarily long message to each of its neighbors. Nodes can perform arbitrary computations between rounds. The complexity of Π, which is typically measured as a function of the number of nodes n, is the number of rounds necessary for all nodes to have computed their output. In the randomized variant of the model, nodes are allowed to output an incorrect solution with probability $n^{-\Omega(1)}$. Throughout the paper, the deterministic setting is the default. We explicitly mention randomness whenever it plays a role.

Since the model was introduced by Linial [11], our understanding of the computational power of Local has grown considerably. A characteristic of Local that was uncovered in recent years is that for some large classes of natural problems, only a few complexities are possible. Defined by Naor and Stockmeyer [13], Locally Checkable Labeling (LCL) problems are problems that can be defined by a set of local rules (what labelings are acceptable in the distance-$O(1)$ neighborhood of any node), on graphs of constant maximum degree $\Delta \in O(1)$. On trees, Chang and Pettie [4] showed that every LCL problem of complexity $n^{o(1)}$ is actually of complexity $O(\log n)$. They also showed that all LCL problems that admit a *randomized* algorithm of complexity $o(\log n)$ can be solved in $O(T_{\mathsf{LLL}})$ time, where T_{LLL} denotes the complexity of the algorithmic *Lovász Local Lemma* problem, an important tool for proving the existence of combinatorial objects through the probabilistic method. On trees and general graphs, Chang, Kopelowitz, and Pettie [3] showed that any LCL problem of deterministic complexity $o(\log n)$ actually has complexity $O(\log^* n)$, where $\log^*$ is the very slowly growing function defined by $\log^*(x) = 0$ when $x \leq 1$ and $\log^*(x) = \log^*(\log x) + 1$ when $x > 1$.

The *sinkless orientation* problem is arguably the simplest LCL problem of deterministic complexity $\Theta(\log n)$ in Local on trees and general graphs [2, 3, 8]. Its randomized complexity on the same graph classes is also fully understood, standing at $\Theta(\log \log n)$. Defined as the problem of orienting edges of the graph s.t. no node of degree 3 or more is a sink (has no outgoing edge), it serves as an important example of a problem that is a *fixed point* in the lower bound technique known as *round elimination* [1]. Intuitively, every T-round algorithm for sinkless orientation where $T < c \log n$ for an appropriate constant c implies the existence of an algorithm of complexity $T - 1$. Repeating the argument, we get that the problem must either be 0-round solvable or require $\Omega(\log n)$ rounds. Ruling out 0-round solvability yields the $\Omega(\log n)$ lower bound. The $\Omega(\log \log n)$ randomized lower bound follows from a structural theorem connecting randomized and deterministic complexities [3].

The Congest model of distributed computing is a restriction of Local where messages are at most $O(\log n)$-bit long [14]. This dramatically increases the complexity of many problems. For example, testing whether the network has diameter 2 or 3 requires $\widetilde{\Omega}(n)$ Congest rounds [9], while the problem is trivially $O(1)$ in Local. The optimal Local algorithms by Ghaffari and Su [8] do not a priori translate well to Congest. In their deterministic $\Theta(\log n)$-round algorithm, each node discovers all cycles of length $O(\log n)$ in its distance-$O(\log n)$ neighborhood, orders them, gives each of them a canonical orientation, and orients its incident edges according to the nearest maximum cycle it found. At the same time, sinkless orientation admits a trivial $O(1)$ algorithm in the Congested Clique model [12], where nodes still compute a solution for some input graph G, but nodes can directly send one another a $O(\log n)$-bit

[*] Also with Inria.
[†] Also with Institut Polytechnique de Paris.
[‡] Also with CISPA Helmholtz Center for Security.



This work is licensed under a Creative Commons Attribution 4.0 International License.
PODC '26, Egham, United Kingdom
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2512-8/26/07
https://doi.org/10.1145/3796701.3815953

message in each round. The problem is easily solved by each node telling the entire network about two of its incident edges. This could suggest that congestion might not be a crucial bottleneck for sinkless orientation, and raises the question: how efficiently can sinkless orientation be solved with congestion constraints, but without all-to-all communication?

Our Results. We show that the sinkless orientation problem has the same complexities in Congest as in Local. Throughout the paper, we call a node v a *sink* iff it has no outgoing edge, and a δ*-sink* if it is a sink and $\deg(v) \geq \delta$.

Theorem 1.1. *For any $\delta \geq 3$, computing an orientation of the edges without any δ-sink admits an algorithm of complexity $O(\log_\delta n)$ (resp. $O(\log_\delta \log n)$) in the deterministic (resp. randomized)* Congest *model.*

The randomized part of the result follows immediately from the deterministic part, due to a *shattering* argument by Ghaffari and Su [8], intuitively reducing solving sinkless orientation with randomness on n-node graphs to solving poly log n-sized instances deterministically. Standard reductions imply the next corollary.

Corollary 1.2. *Let $\varepsilon \in (0, 1/3]$. An orientation where each node v of degree $\deg(v)$ has at least $\lfloor \varepsilon \deg(v) \rfloor$ outgoing and ingoing edges can be computed in $O(\log_{1/\varepsilon} n)$ (resp. $O(\log_{1/\varepsilon} \log n)$) rounds of deterministic (resp. randomized)* Congest.

Our Approach. Our algorithm proceeds in two phases. In a first phase, we compute an orientation of the graph where sinks are separated by a distance of at least $k \in \Theta(\log_\delta n)$. We call such an orientation *k-sinksparse*. In a second phase, all δ-sinks learn some information from their distance-$k/2$ neighborhood. δ-sinks then use this information to reorient some edges in their distance-$k/2$ neighborhood, removing all δ-sinks from the graph in the process.

A natural way to compute a k-sinksparse orientation would be to compute an $O(k, k \log n)$ ruling set[1], and orienting edges towards nodes of the ruling set. However, this approach would take $O(k \log n)$ rounds with the current state-of-the-art [10]. We give a simple procedure for computing a k-sinksparse orientation in $O(k)$ rounds. An important remark is that a ruling set approach computes "too much": in addition to ensuring a minimum distance of k between sinks, it also ensures that all nodes are within some maximum distance of one. Our algorithm guarantees no such thing. Our technique is as simple as each node learning the smallest identifier within distance-k.

To remove the remaining δ-sinks from the graph, each of them performs a BFS of depth $k/2$. The minimum distance between the sinks ensures that the BFS trees of different nodes do not overlap. With k large enough, of order $\Theta(\log_\delta n)$, the distance-$k/2$ neighborhood of every sink is guaranteed to either contain a node of degree $< \delta$, or a cycle. By reorienting at most $O(k)$ edges close to each sink, we remove all δ-sinks from the graph.

Concurrent work. That a sinkless orientation can be computed in $O(\log n)$-rounds of Congest was discovered independently earlier by Fischer and Oshman [6]. The result is part of Fischer's master thesis, which is not publicly available [5].

[1] $S \subseteq V$ is an (α, β)-ruling set iff (i) $\text{dist}(u, v) \geq \alpha$ for any two nodes $u, v \in S$, and (ii) $\forall u \in V$, $\exists v \in S$, $\text{dist}(u, v) \leq \beta$.

2 Our Algorithm

We sketch how to implement the various steps of Algorithm 1 in $O(\log_\delta n)$ rounds of Congest.

Algorithm 1 Computing an orientation without δ-sinks, $\delta \geq 3$

1: Compute a k-sinksparse orientation, for $k = 4\lceil \log_{\delta-1} n \rceil$.
2: Each δ-sink learns about the existence of a nearby node of degree $< \delta$ or a cycle from its distance-$k/2$ BFS.
3: Each δ-sink reorients some edges in its distance-$k/2$ neighborhood, removing all δ-sinks.

Computing a Sinksparse Orientation. As explained previously, the first phase of our algorithm computes a sinksparse orientation.

Theorem 2.1. *k-sinksparse orientation can be computed deterministically in $O(k)$ rounds of* Congest.

Proof. For k rounds, each node broadcasts to its neighbors the smallest identifier it knows about (its own included). At the end of the k rounds, each node knows the smallest identifier in its inclusive distance-k neighborhood. It also knows its distance to that node, which is simply the round in which it learned the identifier, as well as which of its neighbors are at a smaller distance.

For any node v, let m_v denote the smallest identifier in its distance-k neighborhood, and d_v the distance to the node with that identifier. For any two adjacent nodes u and v, we orient the edge uv first according to the order between m_u and m_v, second according to the order between d_u and d_v, third according to the order between ID_u and ID_v, each time towards the smallest value. Any node v s.t. $\text{ID}_v \neq m_v$ is not a sink, since one of its neighbors $u \in N(v)$ is closer to the node of identifier m_v. For this node u either $m_u = m_v$ and $d_u < d_v$, or $m_u < m_v$. So for any two sinks u and v, $\text{ID}_u = m_u$ and $\text{ID}_v = m_v$. Thus, two sinks u and v cannot be within distance k, as that would imply $\max(m_u, m_v) \leq \min(\text{ID}_u, \text{ID}_v)$. □

Note that this is obviously tight, even in Local. This follows from a standard argument assuming the existence of an algorithm of complexity below $c \cdot k$ for some small constant c, and showing that such an algorithm should put two sinks within distance k on some instances.

Removing Remaining Sinks. Crucial to our argument is that every remaining sink should have a node of degree $< \delta$ or a cycle in its distance-$O(\log_\delta n)$ neighborhood. This is guaranteed by the next lemma. The argument is standard, appearing in previous works on sinkless orientation.

Lemma 2.2. *For any node $v \in V$, integers $r \geq 1$ and $\delta \geq 3$, the inclusive distance-r neighborhood of v necessarily contains one of the following: (i) a node of degree $< \delta$, (ii) a cycle, or (iii) $1 + \delta \frac{(\delta-1)^r - 1}{\delta - 2}$ or more nodes. In particular, in an n-node graph, the inclusive distance-$\lceil \log_{\delta-1} n \rceil$ neighborhood of every node necessarily contains a node of degree δ or a cycle.*

All that remains is for each δ-sink in our k-sinksparse orientation to find a nearby node of degree $< \delta$ or a cycle in its distance-$k/2$ neighborhood, and reorient a few edges to obtain a sinkless orientation (see Algorithm 2). We detail these operations in Lemma 2.3.

Algorithm 2 Removing δ-sinks from a k-sinksparse orientation

Assumption: each sink has a node of degree $< \delta$ or a cycle within distance $k/2$.

1: Each δ-sink s learns whether its distance-$k/2$ neighborhood contains a node t of degree $< \delta$. If so, the path between s and one such node t is reoriented towards t.
2: Each remaining δ-sink s finds a node t in its distance-$k/2$ neighborhood s.t. either t has two neighbors t' and t'' that are closer to s, or t has a neighbor t' at equal distance to s.
3: A cycle involving t and nodes at equal or lower distance from the sink s is found, and oriented consistently. The path from the sink to the cycle is oriented towards the cycle.

Lemma 2.3. *Given a k-sinksparse orientation, and assuming that each δ-sink in that orientation has a node of degree $< \delta$ or a cycle in its distance-$k/2$ neighborhood, $O(k)$ rounds of* Congest *suffice to transform the orientation into an orientation without δ-sinks.*

Proof sketch. Each δ-sink s performs a BFS of depth-$k/2$. To find a node of degree $< \delta$ in its BFS tree, s performs the following aggregation: for k rounds, each node in the BFS sends to its parent in the BFS the identifier of the node of smallest degree it knows about, together with that node's degree (if needed, breaking ties by depth in the tree, and identifier). After $k/2$ rounds, s knows whether a node of degree $< \delta$ exists in the BFS tree. If so, s broadcasts the identifier of such a node to its BFS. Every node on a shortest path between the chosen node of degree $< \delta$ and the sink s knows that it is on that shortest path, as they relayed the low degree node's identifier to s. All edges on that shortest path are flipped, and the original sink is no longer a sink. The node of degree $< \delta$ might have become a sink, but is of low degree.

If the distance-$k/2$ neighborhood of the δ-sink s contains no node of degree $< \delta$, then it contains a cycle. It follows that there either exists a node t with a neighbor of equal depth in the BFS tree, or a node t with two neighbors of depth lower than itself in the BFS tree. Each node in the BFS tree can learn whether it is in one of these two scenarios by exchanging its depth in the BFS tree with its neighbors. The sink learns about the existence of such a node by aggregation as we did before for the node of low degree.

The idea in both cases is to find the highest depth common ancestor to two nodes of equal depth in the BFS tree. When t has a neighbor t' of equal depth in the tree, we consider the shortest paths from the sink to t and t'. Taking the last common node v on those two paths, the combination of the shortest paths between v and t, v and t', and the edge tt', forms a cycle. We orient that cycle consistently and reorient a shortest path from the sink s to that cycle (see Fig. 1). When t has two neighbors t' and t'' of lower depth in the BFS tree, we instead compute the shortest paths from s to those two nodes, and consistently orient the cycle consisting of the edges tt', tt'', and the shortest paths between t', t'' and their common ancestor v of highest depth in the BFS tree. □

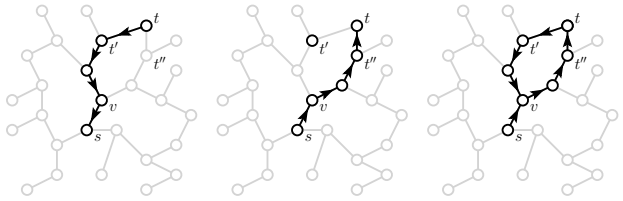


Figure 1: Re-orientation of edges creating an oriented cycle near a sink s as in Algorithm 2.

3 Corollaries

In this section, we explain how some additional results follow immediately from our deterministic Congest algorithm for sinkless orientation.

Randomized CONGEST. The $O(\log_\delta \log n)$ algorithm in randomized Congest follows straightforwardly from the deterministic algorithm. Ghaffari and Su [8] gave a simple $O(1)$ randomized procedure to reduce the problem of computing sinkless orientation on n-node graphs to solving smaller instances of size poly $\log n$, but deterministically. Their procedure trivially also works in Congest, as it only consists of each edge being randomly marked with probability $1/4$, randomly orienting each marked edge, and erasing the orientation of edges incident to a node with more than $\delta/2$ incident marked edges. Some small adjustments to our algorithm are needed for use in the second, deterministic part of their algorithm, but these are minor: in addition to the nodes of degree $< \delta$, nodes that obtained an outgoing edge during the randomized phase can also be used as sinks. They are easily integrated in our algorithm as a third option to remove sinks after computing a sinksparse orientation.

More balanced orientations. A standard extension of the sinkless orientation result in Local is to compute an orientation that is both sinkless and sourceless, i.e., where each node of degree $\geq \delta$ has at least both 1 incoming and 1 outgoing edge. Another standard extension is to have every node v have at least $\max(\varepsilon \deg(v), 1)$ outgoing edges and $\max(\varepsilon \deg(v), 1)$ incoming edges, for $\varepsilon \leq 1/3$. These extensions follow from standard reductions to sinkless orientation that also efficiently work in Congest. The stronger result is obtained by splitting each node v into $\varepsilon \deg(v)$ virtual nodes of degree $1/\varepsilon$, before computing a sinkless and sourceless orientation of the resulting graph. For more details, see notably [7, Lemma 2 and Corollary 5], and [8, Lemma A.9 and A.10 in arxiv version].

Acknowledgments

Funded by the European Union. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them. This work is supported by ERC grant OLA-TOPSENS (grant agreement number 101162747) under the Horizon Europe funding programme. The author changed affiliation from CISPA to Télécom SudParis during the preparation of this work. The author would like to thank Yann Bourreau, Sebastian Brandt, and Ananth Narayanan for interesting discussions on the topic of this paper.

References

[1] Alkida Balliu, Janne H. Korhonen, Fabian Kuhn, Henrik Lievonen, Dennis Olivetti, Shreyas Pai, Ami Paz, Joel Rybicki, Stefan Schmid, Jan Studený, Jukka Suomela, and Jara Uitto. 2023. Sinkless Orientation Made Simple. In *Proc. SIAM Symp. on Simplicity in Algorithms (SOSA)*. SIAM, 175–191. doi:10.1137/1.9781611977585.CH17

[2] Sebastian Brandt, Orr Fischer, Juho Hirvonen, Barbara Keller, Tuomo Lempiäinen, Joel Rybicki, Jukka Suomela, and Jara Uitto. 2016. A lower bound for the distributed Lovász local lemma. In *Proc. ACM Symp. on Theory of Computing (STOC)*. ACM, 479–488. doi:10.1145/2897518.2897570

[3] Yi-Jun Chang, Tsvi Kopelowitz, and Seth Pettie. 2019. An Exponential Separation between Randomized and Deterministic Complexity in the LOCAL Model. *SIAM Journal on Computing* 48, 1 (2019), 122–143. doi:10.1137/17M1117537

[4] Yi-Jun Chang and Seth Pettie. 2019. A Time Hierarchy Theorem for the LOCAL Model. *SIAM Journal on Computing* 48, 1 (2019), 33–69. doi:10.1137/17M1157957

[5] Orr Fischer. 2016. Topics in distributed computing and simultaneous multiparty communication complexity. (2016). master thesis.

[6] Orr Fischer. 2024. Private communication.

[7] Mohsen Ghaffari, Juho Hirvonen, Fabian Kuhn, Yannic Maus, Jukka Suomela, and Jara Uitto. 2020. Improved distributed degree splitting and edge coloring. *Distributed Computing* 33, 3-4 (2020), 293–310. doi:10.1007/S00446-018-00346-8

[8] Mohsen Ghaffari and Hsin-Hao Su. 2017. Distributed degree splitting, edge coloring, and orientations. In *Proc. ACM-SIAM Symp. on Discrete Algorithms (SODA)*. SIAM, 2505–2523.

[9] Stephan Holzer and Roger Wattenhofer. 2012. Optimal distributed all pairs shortest paths and applications. In *Proc. ACM Symp. on Principles of Distributed Computing (PODC)*. ACM, 355–364. doi:10.1145/2332432.2332504

[10] Fabian Kuhn, Yannic Maus, and Simon Weidner. 2018. Deterministic Distributed Ruling Sets of Line Graphs. In *Proc. Int. Coll. on Structural Information and Communication Complexity (SIROCCO) (LNCS, Vol. 11085)*. Springer, 193–208. doi:10.1007/978-3-030-01325-7_19

[11] Nati Linial. 1992. Locality in Distributed Graph Algorithms. *SIAM J. Comput.* 21, 1 (1992), 193–201.

[12] Zvi Lotker, Boaz Patt-Shamir, Elan Pavlov, and David Peleg. 2005. Minimum-Weight Spanning Tree Construction in $O(\log \log n)$ Communication Rounds. *SIAM Journal on Computing* 35, 1 (2005), 120–131. doi:10.1137/S0097539704441848

[13] Moni Naor and Larry J. Stockmeyer. 1995. What Can be Computed Locally? *SIAM Journal on Computing* 24, 6 (1995), 1259–1277. doi:10.1137/S0097539793254571

[14] David Peleg. 2000. *Distributed Computing: A Locality-Sensitive Approach*. SIAM.